AN INTERNET OF THINGS CASE STUDY

ON

WiFi CSI-Based Device-Free Human Occupancy Counting

Using ESP32A Case Study in Layered IoT Architecture, Protocols, and Security.

Submitted in Partial Fulfillment of the Requirements for
the Degree of

Bachelor of Engineering in IT Engineering
Under Pokhara University

Submitted by:

Navanit Sharma (231519)

Submitted to:

Er. Rishi K. Marseni

Submission Date:

13 July 2026

6th Semester

Nepal College of Information Technology

Affiliated to Pokhara University

Contents

List of Figures

LIST OF TABLES

List of Tables

1 Introduction

The Internet of Things (IoT) refers to the network of physical objects embedded with electronics, sensors, and software that collect and exchange data over the internet, bridging the physical and digital worlds. Over the past decade, IoT has evolved from simple telemetry into intelligent, self-adapting networks that sense, reason, and act on real-world conditions with minimal human intervention. Among the many emerging IoT applications, human presence and occupancy sensing has become critical for smart buildings, energy management, security, and space utilisation. The ability to know how many people are in a room enables demand-driven HVAC control, adaptive lighting, emergency evacuation planning, and efficient use of building resources.

This case study examines a WiFi Channel State Information (CSI)-based human counting system built around the low-cost Espressif ESP32 microcontroller. Instead of relying on cameras or dedicated presence sensors, this system exploits the fact that the human body absorbs, reflects, and scatters ambient WiFi signals as it moves through a room, subtly distorting the amplitude and phase of every OFDM subcarrier that reaches the receiver. By capturing and analysing these distortions, a pair of ESP32 modules can estimate how many people are present entirely device-free and without ever recording an identifiable image.

The topic is highly relevant today. WiFi sensing has matured from a research curiosity into standardised technology: the IEEE approved the 802.11bf WLAN Sensing amendment in 2024, making sensing a first-class feature of ordinary WiFi hardware. Simultaneously, growing concerns about camera-based surveillance have driven strong interest in privacy-preserving sensing alternatives that never capture facial features or identifiable footage. This topic was selected because it combines embedded systems, signal processing, cloud computing, and IoT security into a coherent real-world system, directly applying the layered architecture, protocol stack, and security principles covered throughout this course.

2 Background Study

Counting people in indoor spaces has traditionally used one of three approaches. Passive infrared sensors are cheap and power-efficient but only report binary motion, unable to distinguish one person from a crowd. Camera-based computer vision systems are accurate but raise serious privacy concerns, require direct line of sight, and demand considerable computing power. Device-based approaches such as Bluetooth or WiFi probe scanning require occupants to carry smartphones, which is increasingly impractical as modern phones randomise their MAC addresses for privacy reasons.

WiFi CSI sensing avoids all three shortcomings. CSI captures fine-grained per-subcarrier amplitude and phase measurements of the wireless channel, which change measurably whenever a person moves, breathes, or simply occupies a room, enabling a wide family of device-free recognition tasks. Studies have confirmed that CSI-based counting achieves accuracy competitive with vision-based systems while preserving privacy, though challenges around environment dependence and multi-person separation remain open research problems.

Early CSI research required expensive laptops fitted with modified Intel 5300 network cards, limiting deployment outside laboratory settings. This changed dramatically when community tools exposed CSI directly from the commodity ESP32, a microcontroller costing only a few dollars. Recent work has shown that a single ESP32 pair can replace an entire laptop-based CSI rig for crowd counting, dramatically lowering both cost and physical footprint. Other ESP32 systems have demonstrated simultaneous counting and localisation with sub-person median absolute error.

An IoT-based architecture is essential because a CSI sensor in isolation has limited value. Its readings must be transmitted, stored, visualised, and acted upon in real time. Without networking, cloud storage, and a dashboard, a raw occupancy count sitting inside a microcontroller cannot inform facility managers,

trigger HVAC automation, or feed historical analytics. This project therefore situates WiFi CSI sensing as the device-layer innovation within a complete IoT stack.

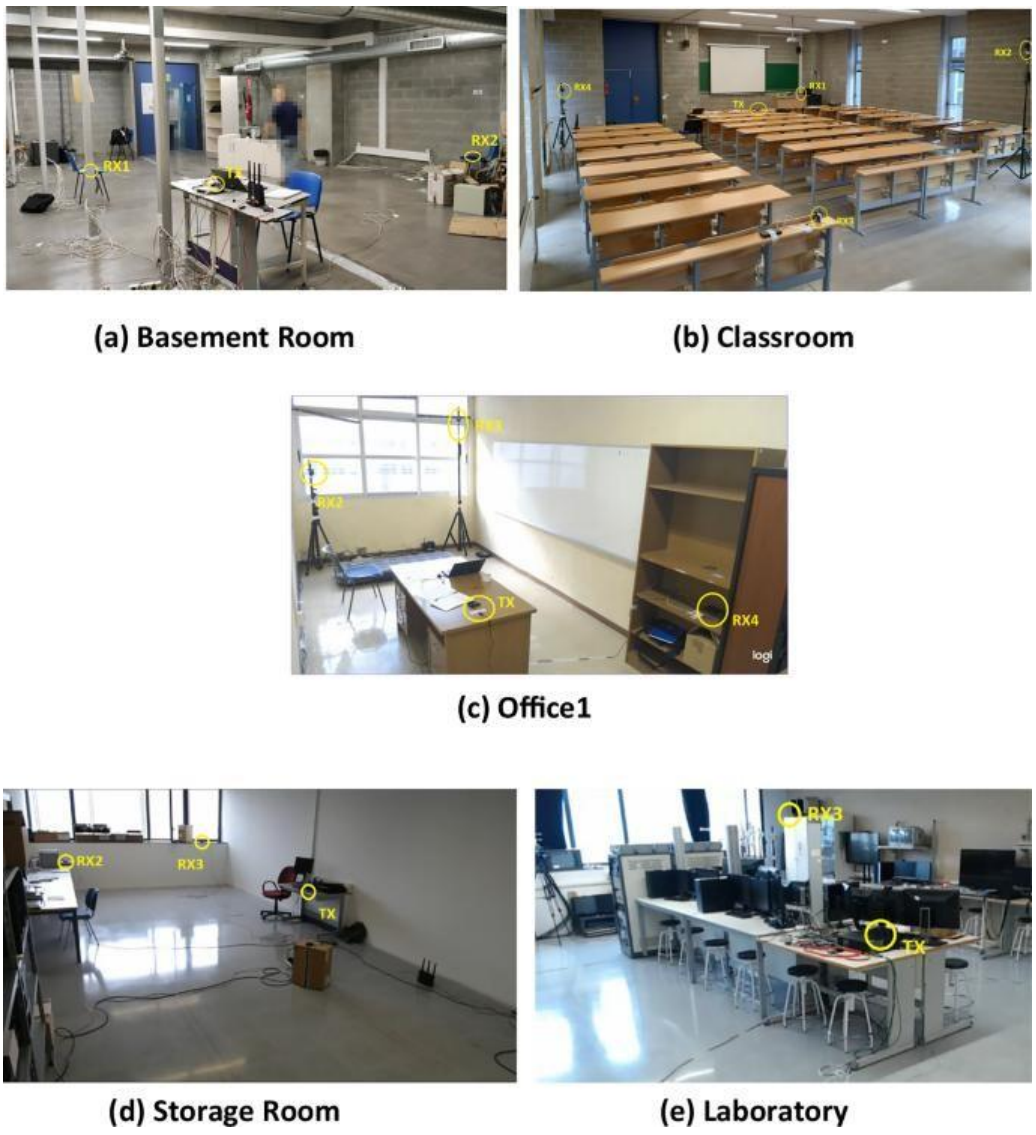

Figure 1: CSI measurement environments used in prior studies showing typical indoor deployment scenarios including rooms, corridors, and open spaces.

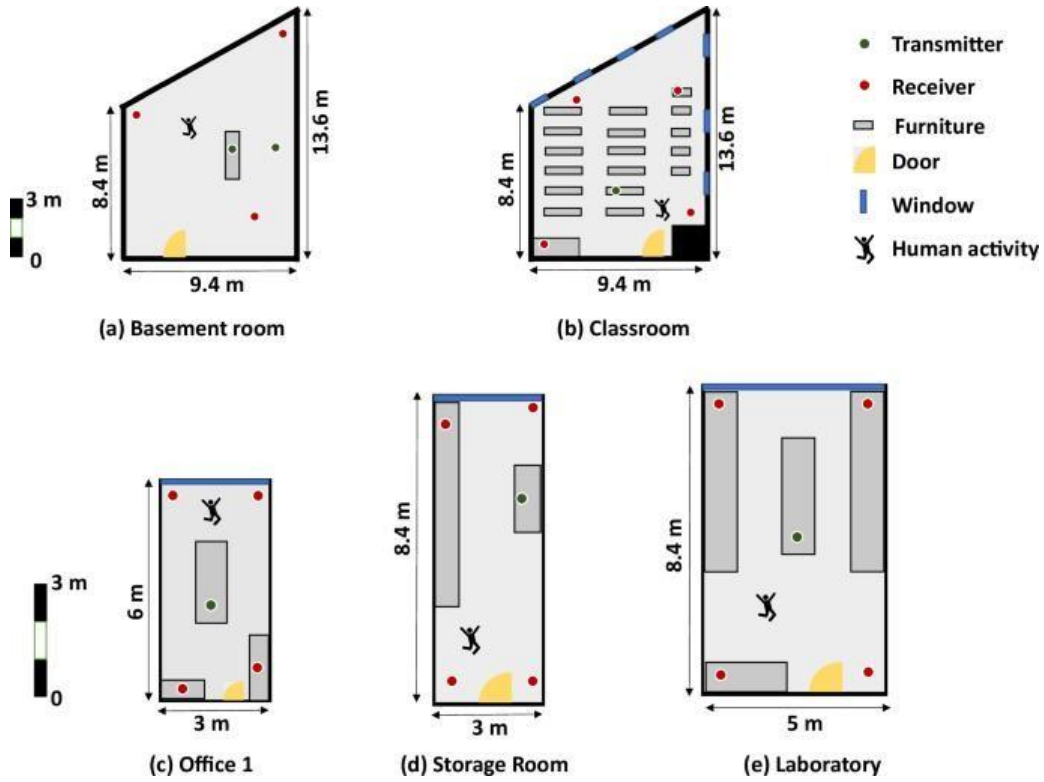

Figure 2: Measurement environments covering basement, classroom, office, storage, and laboratory settings used to validate CSI-based sensing across diverse conditions.

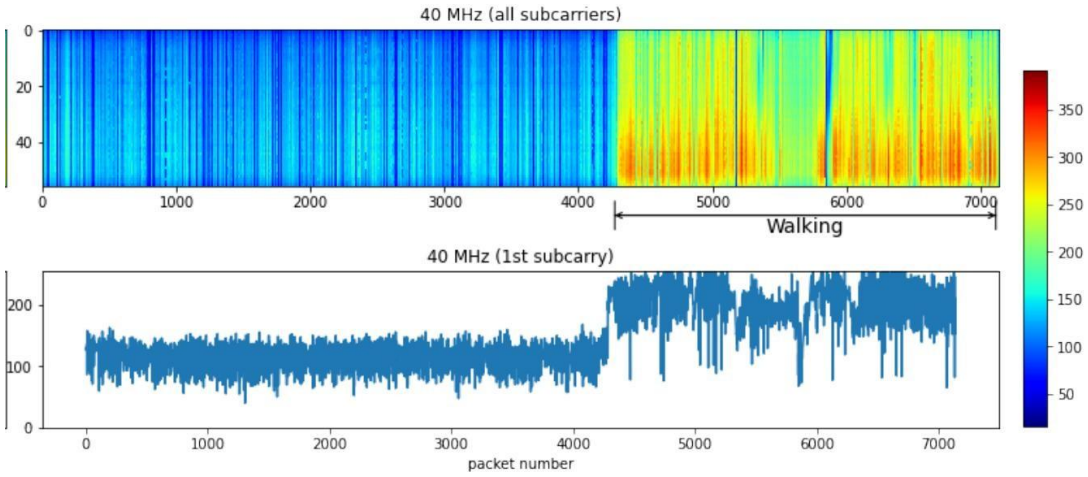

Figure 3: CSI amplitude heatmap at 40 MHz bandwidth captured during walking motion, demonstrating how human movement modulates per-subcarrier signal strength over time.

4 SYSTEM ARCHITECTURE

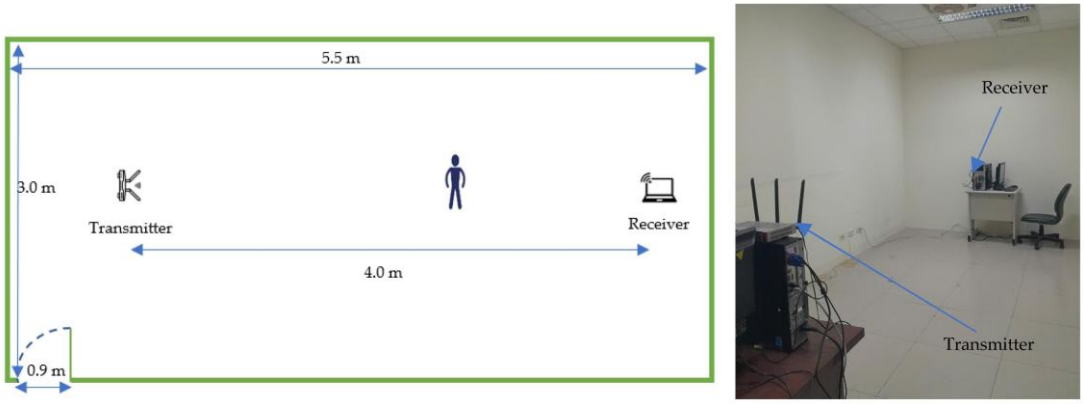

Figure 4: CSI walking time-series heatmap showing amplitude variations across subcarriers during continuous pedestrian movement through the sensing zone.

3 Problem Statement

Many indoor spaces including classrooms, meeting rooms, offices, and libraries have no reliable, lowcost method of knowing how many people are inside at any moment. Manual headcounts are slow and error-prone. PIR sensors cannot distinguish one occupant from ten. Camera-based counting is expensive, computationally heavy, and privacy-invasive since it necessarily captures identifiable images of every occupant.

This project addresses that gap by using WiFi CSI captured from a pair of ESP32 microcontrollers to estimate the number of people in a monitored room without cameras, wearables, or any device carried by occupants. The problem matters because accurate real-time occupancy data underpins energy-efficient HVAC and lighting control, safe evacuation planning, capacity compliance, and space utilisation analytics for organisations seeking to reduce operating costs.

The direct beneficiaries include facility and energy managers who can automate climate and lighting based on actual usage, safety officers who need real-time occupancy for emergency evacuation coordination, and building occupants themselves who gain the benefits of an intelligent environment without being filmed or individually tracked.

4 System Architecture

The system follows the classic three-layer IoT architecture taught in Unit 2: a Device Layer that senses the physical world, a Network Layer that transports data, and a Cloud Layer that stores, processes, and presents information to end users.

Device Layer. Two ESP32 microcontrollers form a WiFi transmitter-receiver pair. One operates as a Soft-Access-Point continuously broadcasting standard 802.11n frames; the other operates as a Station that captures CSI from every received frame using Espressif's CSI-extraction APIs. Unlike a conventional IoT device layer built from discrete environmental sensors, this project needs no dedicated sensor at all since the WiFi radio itself is the sensing instrument. A Raspberry Pi acts as an optional edge gateway running feature extraction and the counting model locally. A relay module driven by a GPIO pin closes the automation loop by switching room lighting or fans based on detected occupancy levels.

Network Layer. The gateway communicates with the cloud over WiFi using HTTP and REST API calls to POST occupancy counts with timestamps. A WebSocket channel enables real-time dashboard updates

without polling overhead. MQTT is available as a lower-overhead alternative for multi-room scaling, and CoAP is considered for future battery-powered sensing nodes.

4 SYSTEM ARCHITECTURE

Cloud Layer. The backend is deployed on AWS EC2 running a FastAPI service, or alternatively on Firebase for rapid prototyping. The cloud persists every reading in a database, exposes REST endpoints for dashboard queries, and pushes live updates over WebSocket. Each layer can be upgraded independently without redesigning the others.

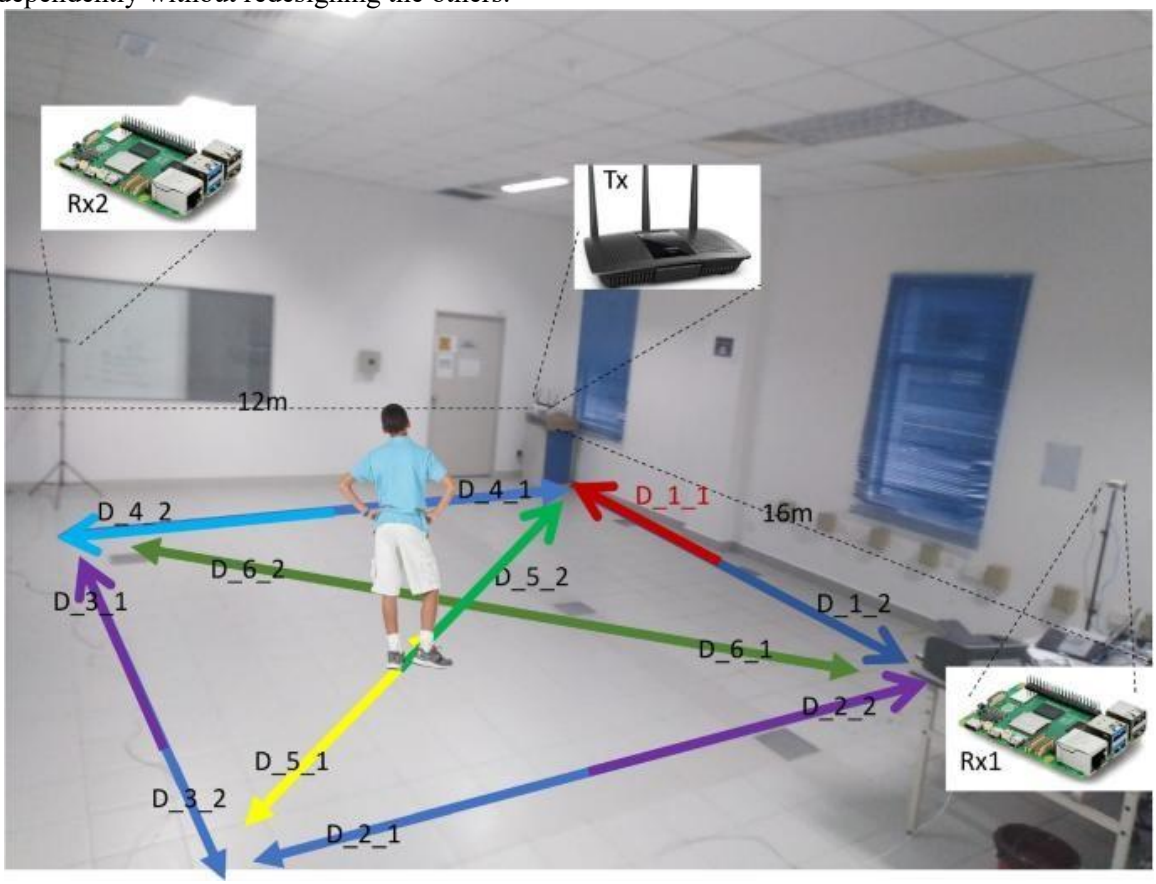

Figure 5: Three-layer IoT architecture of the WiFi CSI human-counting system showing the device, network, and cloud layers with ESP32 transmitter-receiver pair.

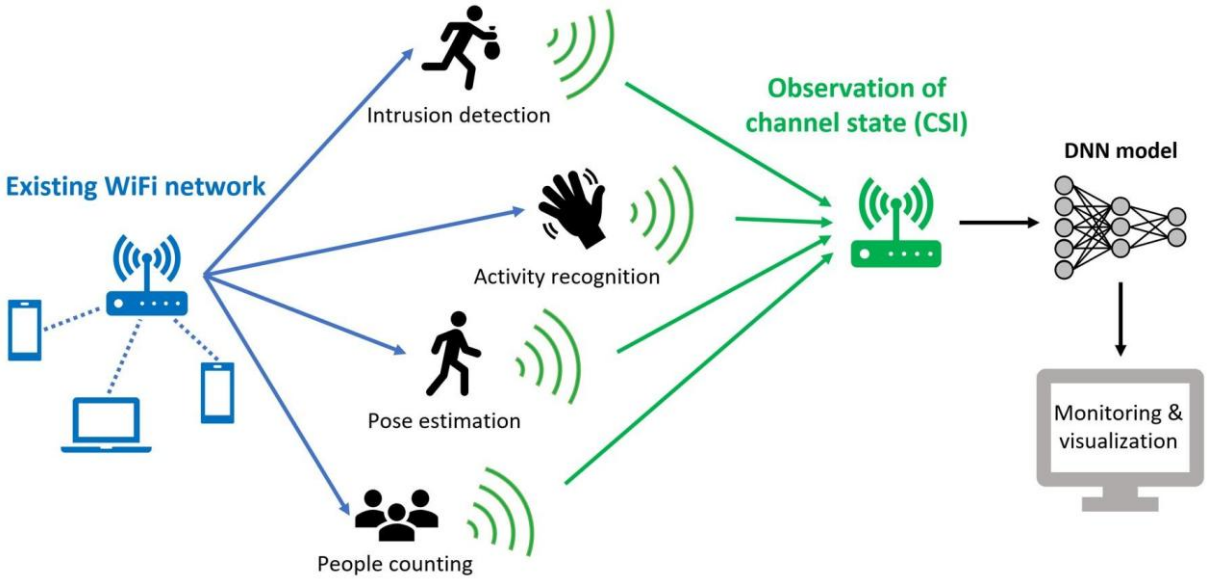

Figure 6: System block diagram showing the complete CSI-based human activity recognition pipeline from signal capture through feature extraction to classification and actuation.

5 Hardware Components

ESP32 Microcontroller. The ESP32 is a dual-core Xtensa LX6 processor running at 240 MHz with integrated 802.11 b/g/n WiFi and Bluetooth, 520 KB of internal SRAM, and 34 GPIO pins supporting ADC, DAC, SPI, I2C, and UART interfaces. Its advantages for this project are threefold: it costs only a few dollars per unit, it exposes raw CSI directly through its WiFi driver without any firmware patching, and it is small and low-power enough for unobtrusive ceiling or wall mounting. Recent ESP32-S3 variants with directional antennas have demonstrated through-wall human activity recognition at distances beyond 18 metres.

Sensor - The WiFi Channel. The sensor is not a discrete electronic component but the WiFi propagation channel itself. Its working principle is multipath fading: every wall, piece of furniture, and human body reflects, absorbs, and scatters part of the transmitted signal. The receiver observes the combined interference pattern as amplitude and phase values across 52 OFDM subcarriers. An ESP32-based crowd counting system has demonstrated a median absolute counting error of only 0.35 people with up to five occupants.

Actuators. A relay module switches mains-powered loads such as room lighting or ceiling fans under GPIO control. When occupancy drops to zero for a configurable timeout, the relay de-energises the load, closing the sense-then-act loop. The same relay approach extends to pumps, motors, or larger HVAC contactors in other IoT applications.

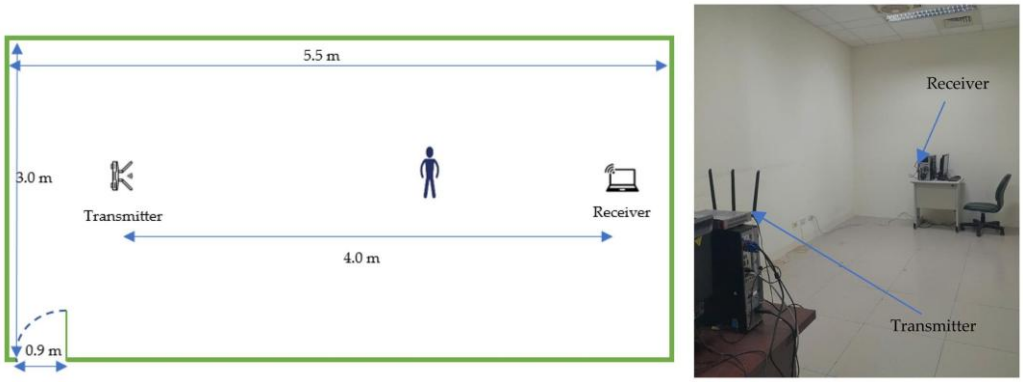

Figure 7: Indoor room geometry showing transmitter and receiver placement for optimal CSI-based occupancy sensing coverage.

6 SOFTWARE COMPONENTS

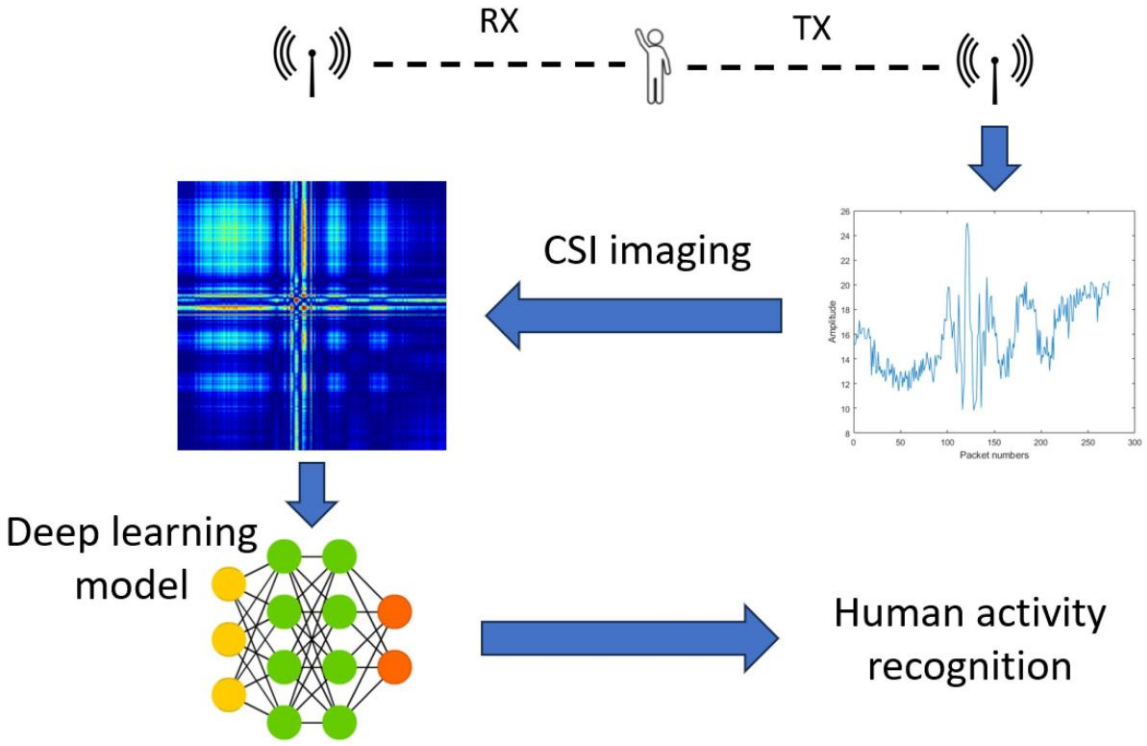

Figure 8: Sensor connection diagram: indoor measurement setup showing the ESP32 transmitter-receiver pair with a person in the sensing area forming the WiFi CSI sensing link.

Figure 9: ESP32 development board used as the core microcontroller for CSI capture, feature extraction, and GPIO-based relay actuation.

6 Software Components

Programming Languages. The ESP32 firmware is written in C using the ESP-IDF framework since CSI extraction requires low-level access to the WiFi driver stack. All downstream processing including CSI denoising, feature extraction, and the classification model is written in Python, leveraging its mature ecosystem of NumPy, scikit-learn, and TensorFlow Lite for efficient numerical computation.

Frameworks. The backend REST API is built with FastAPI for its speed, automatic OpenAPI documen-

tation, and native asynchronous support ideal for handling frequent sensor uploads alongside WebSocket connections. Flask is a viable simpler alternative for early prototyping. The machine learning model uses TensorFlow Lite Micro for on-device inference directly on the ESP32.

Database. SQLite suffices during development for logging timestamped occupancy records per room. As the system scales to multiple rooms and buildings, PostgreSQL is preferred for its concurrency control, indexing performance, and time-series query support.

Dashboard. The web dashboard is built with HTML and JavaScript using Chart.js to render the live occupancy trend as a continuously updating line chart. A WebSocket client receives new readings pushed by the backend and updates both the chart and headline count without any page refresh.

Cloud. The complete backend, database, and dashboard run on a single AWS EC2 instance. Azure and Google Cloud Compute Engine offer equivalent hosting. Firebase provides a serverless path with minimal operations overhead for rapid prototyping.

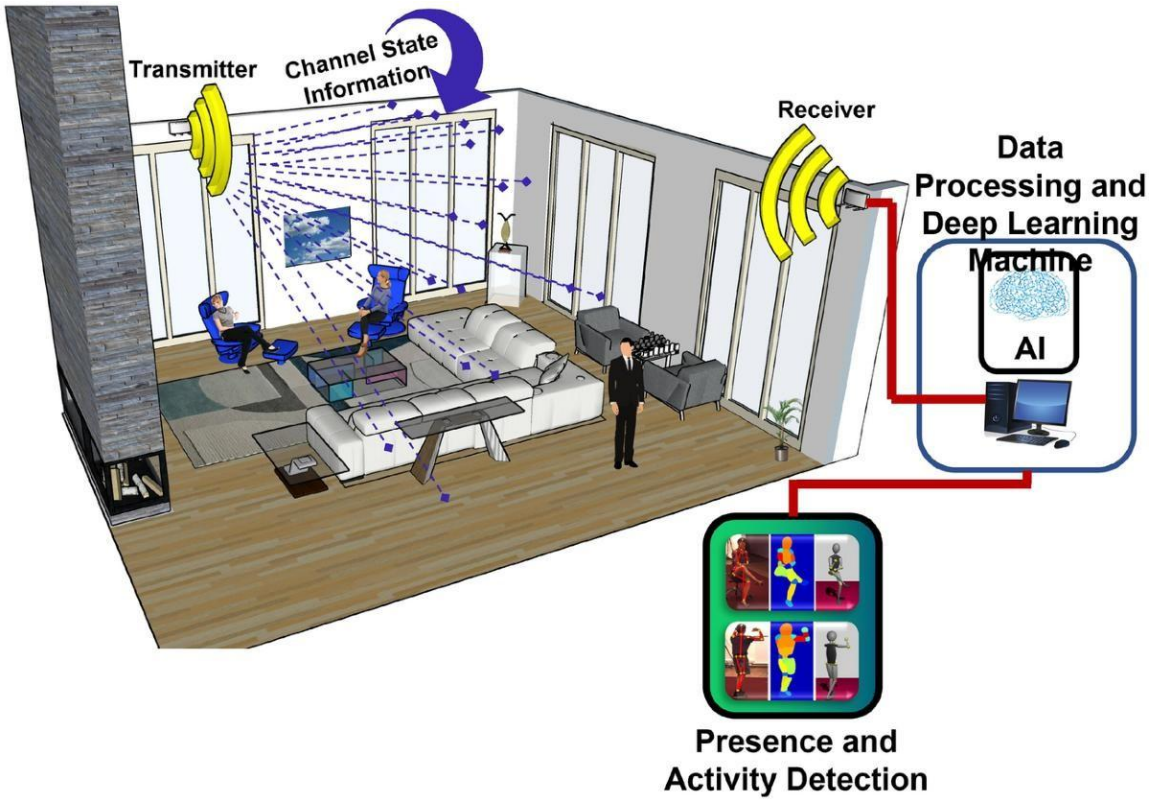

Figure 10: Deep learning pipeline for WiFi CSI-based presence and activity detection showing data preprocessing, model architecture, and classification outputs used in the occupancy counting dashboard.

7 Communication Protocols

Four communication protocols are relevant. HTTP used through a REST API is simple and universally supported by cloud platforms, but its text-based headers and per-request TCP handshake make it comparatively heavy for frequent small payloads. MQTT is a lightweight publish-subscribe protocol designed for constrained and unreliable networks; a single broker can efficiently fan out occupancy data to the dashboard, an HVAC controller, and a mobile app simultaneously. CoAP mirrors the REST requestresponse model but runs over UDP with compact binary headers, making it attractive for battery-powered edge nodes. WebSocket maintains a persistent full-duplex connection ideal for pushing real-time dashboard updates the instant a new count is computed.

8 DATA FLOW ANALYSIS

Table 1: Comparison of communication protocols relevant to the project.

Protocol	Speed	Reliability	Power Use	Scalability
HTTP / REST	Moderate	High (TCP, synchronous)	High	Moderate
MQTT	High	High (QoS 0/1/2)	Low	High (broker fan-out)
CoAP	High	Moderate (UDP)	Very low	High
WebSocket	Very high	High (persistent TCP)	Moderate	Moderate

For this prototype, HTTP and REST were selected as the primary uplink protocol because they integrate directly with the FastAPI backend built in Lab 2 without requiring additional broker infrastructure.

WebSocket was added purely for the dashboard live update feature. MQTT is recommended as the upgrade path when scaling beyond a few rooms.

8 Data Flow Analysis

A single occupancy reading flows from the physical WiFi channel to the end user through a structured pipeline. The AP-side ESP32 continuously transmits standard frames; the STA-side ESP32 captures the resulting CSI for every received frame, producing a stream of amplitude and phase values across 52 OFDM subcarriers at approximately 50 Hz. This raw stream is denoised through moving average and low-pass filtering, then reduced to a compact feature vector containing variance, inter-subcarrier correlation, and spectral energy features computed over a two-second sliding window. The feature vector feeds a trained Random Forest classifier to produce a single integer occupancy count.

The count together with a room identifier and timestamp is POSTed as JSON to a FastAPI REST endpoint on AWS EC2, directly applying the API design pattern from Lab 2. The backend writes the reading to a PostgreSQL database and simultaneously pushes it over WebSocket to every connected dashboard client. The dashboard styled with Chart.js from Lab 3 appends the new point to the live occupancy chart without page reload. If the count remains zero for a configured duration, the backend signals the GPIO relay to switch off room lighting.

9 SECURITY ANALYSIS

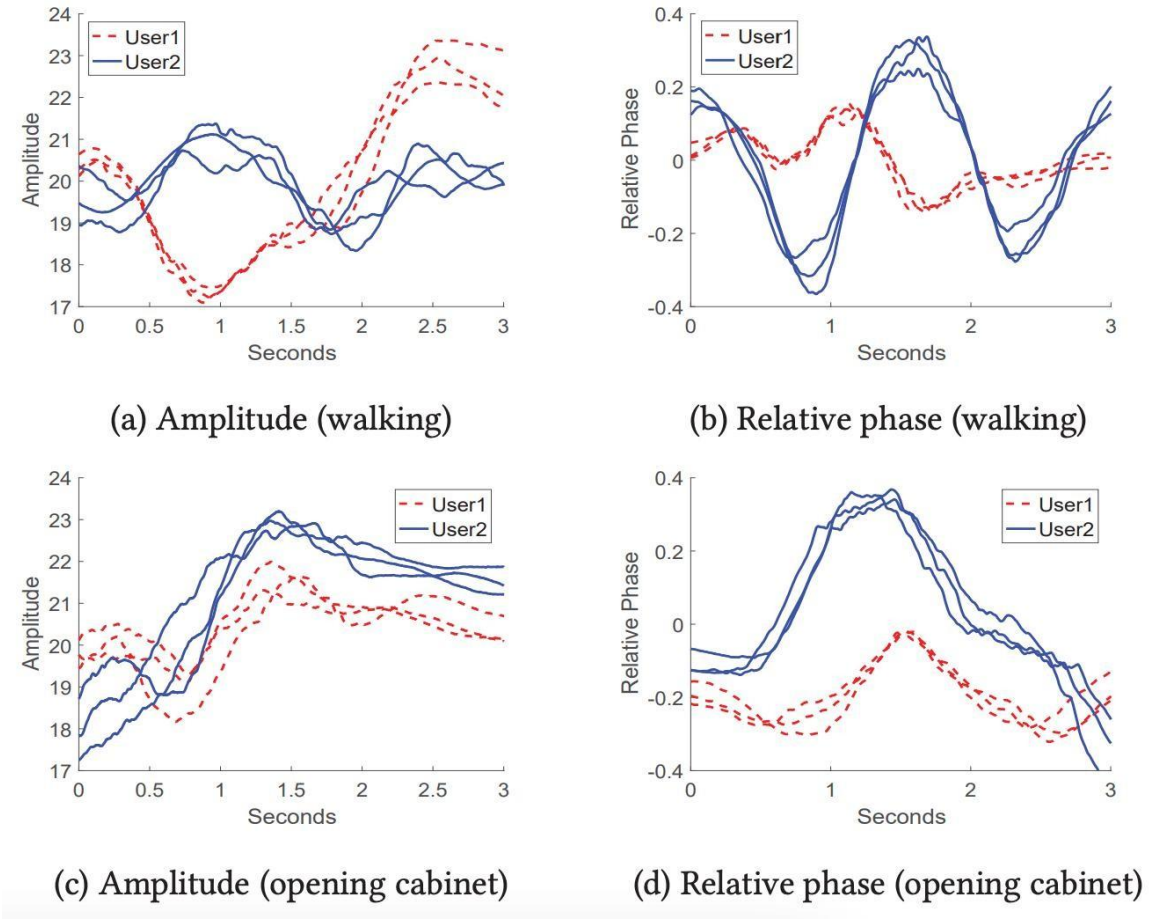

Figure 11: CSI amplitude and relative phase measurements captured during walking motions around a cabinet, illustrating the data flow from raw WiFi channel readings through signal processing to occupancy classification.

This complete chain from sensor to microcontroller to WiFi to REST API to cloud to database to dashboard to actuator directly mirrors the end-to-end IoT pipeline exercised in Lab 5.

9 Security Analysis

CSI-based occupancy sensing introduces both the standard IoT threats covered in Unit 6 and a unique physical-layer risk. Unauthorised access to the dashboard or REST API would let an outsider view or modify live occupancy data. Data tampering could inject fabricated counts to mask an intrusion or falsely trigger automation. Replay attacks could resend a previously captured empty-room reading to disable energy-saving logic at an incorrect time. Password attacks against device credentials remain a risk wherever authentication is implemented. Uniquely, the CSI channel itself forms a passive attack surface since CSI data typically travels unencrypted at the physical layer, allowing a nearby eavesdropper to silently reconstruct occupancy patterns.

Several countermeasures are deployed. All gateway-to-cloud traffic uses HTTPS and TLS 1.3 to prevent on-path tampering. The REST API authenticates each gateway using a unique API key. The dashboard requires username and password authentication. Stored occupancy history is encrypted at rest in the database. WPA2 security on the underlying WiFi network restricts physical-layer proximity. Ratelimiting and timestamp validation on the API prevent brute-force and replay attacks by rejecting stale payloads.

11 CHALLENGES AND LIMITATIONS

10 Mapping with Course Labs

Every stage of this project maps to a specific laboratory exercise from the course. Lab 1 on hosting a web server using AWS EC2 directly provides the infrastructure running the FastAPI backend and dashboard, including security group configuration for HTTPS ingress. Lab 2 on REST API design supplies the JSON POST endpoint that accepts occupancy readings with room ID, count, and timestamp, reusing the same FastAPI patterns practised in class. Lab 3 on dashboard development contributes the HTML, JavaScript, and Chart.js frontend skeleton, extended with a WebSocket client for real-time live updates on the occupancy trend chart. Lab 4 on embedded devices covers flashing the ESP32 firmware, configuring GPIO pins for the relay, reading serial CSI data, and deploying the TensorFlow Lite Micro model. Lab 5 on end-to-end IoT integration connects the complete pipeline from physical WiFi channel to ESP32 capture to gateway processing to REST API upload to cloud database to dashboard visualisation to actuator response.

Table 2: Mapping of the project to the course laboratory exercises.

Lab	Core Concept	Connection to This Project
Lab 1	Hosting web server on AWS EC2	EC2 provisioning hosts FastAPI backend and dashboard with HTTPS security groups.
Lab 2	REST API design	JSON POST endpoint for occupancy readings reuses FastAPI patterns from class.
Lab 3	Dashboard development	HTML/JavaScript/Chart.js skeleton extended with WebSocket for real-time updates.
Lab 4	Embedded devices	Flashing ESP32, GPIO configuration, serial data reading, and relay control.
Lab 5	End-to-end IoT	Full sensor-to-cloud-to-actuator pipeline integrated as taught in Lab 5.

These mappings demonstrate that the Device-Network-Cloud architecture and the protocol, dashboard, and hosting skills developed throughout the course combine directly into a single coherent deployed system rather than remaining five disconnected exercises.

11 Challenges and Limitations

Despite its advantages, the system faces several practical challenges. Internet dependency means realtime dashboards fail when the cloud connection drops, though local relay automation can continue operating offline. Sensor accuracy degrades when furniture is rearranged or crowd size exceeds training levels since overlapping human bodies produce increasingly similar multipath distortions. The physical-layer eavesdropping risk described in the security section cannot be eliminated by application-layer measures alone. Power consumption exceeds that of a duty-cycled PIR sensor because both ESP32 radios must remain continuously active to sustain the CSI sampling rate. Cloud cost scales linearly with each monitored room due to increasing API traffic and compute requirements. Co-channel interference between neighbouring ESP32 pairs on the same WiFi channel can corrupt CSI readings unless channels or time slots are carefully coordinated. Every significant room layout change requires model recalibration, demanding expertise that a binary PIR sensor would never need.

13 PERSONAL REFLECTION

12 Future Improvements

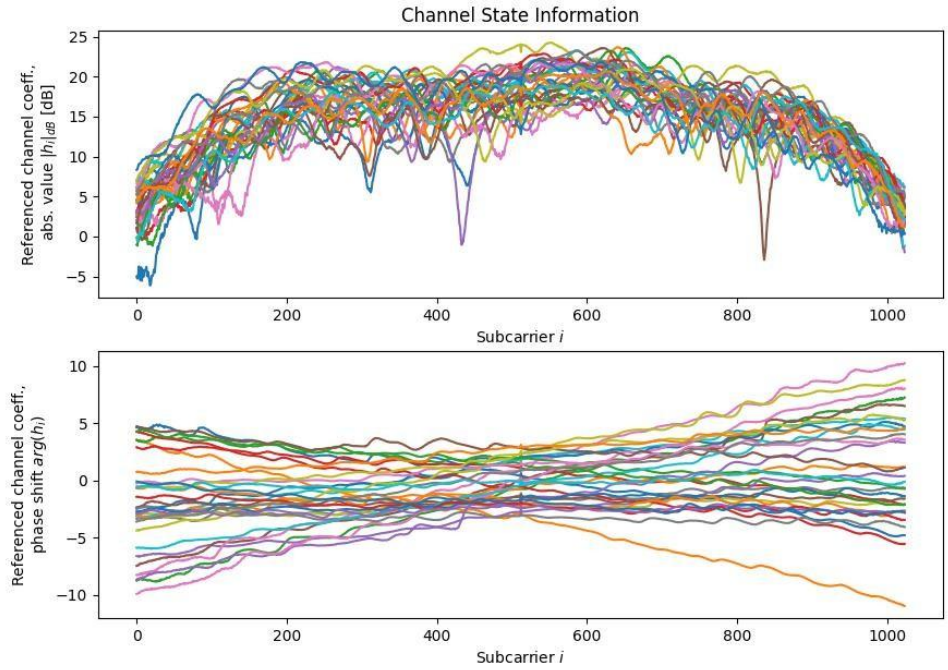

Figure 12: CSI-based imaging and deep learning activity recognition framework representing a future direction for enhancing occupancy classification accuracy beyond current Random Forest approaches.

Several directions could meaningfully advance the system. Deeper convolutional or recurrent neural networks trained directly on CSI spectrograms rather than hand-crafted features could improve counting accuracy for larger and more overlapping crowds. Running quantised int8 models directly on the ESP32S3 rather than on a separate Raspberry Pi would reduce both latency and dependency on constant

network connectivity. The arrival of 5G networking could support denser building-wide deployments through higher-bandwidth, lower-latency backhaul between multiple gateways and the cloud. Digital twins built from continuous occupancy data would let facility managers simulate HVAC or lighting strategies against a virtual building model before physical application. Predictive analytics using historical occupancy patterns with time-series forecasting could pre-condition building systems ahead of expected occupancy surges. Blockchain-based logging could provide tamper-evident audit trails for compliance-sensitive settings such as fire-safety capacity reporting. Most significantly, the IEEE 802.11bf WLAN Sensing standard means CSI-like sensing may soon become a native feature of ordinary consumer WiFi chipsets.

13 Personal Reflection

This case study deepened my understanding that an IoT system is fundamentally more than the sum of its sensors. The layered Device-Network-Cloud model taught throughout this course had to be applied concretely at every step from flashing ESP32 firmware to designing REST API endpoints, wiring a relay module, and building a live web dashboard with Chart.js and WebSocket. I gained hands-on familiarity with FastAPI route design, real-time data push mechanisms, and the practical trade-offs between HTTP, MQTT, and CoAP that are straightforward to describe in lecture slides but only become truly intuitive when justifying a choice for a real device with real constraints. The security unit proved especially valuable here because WiFi CSI sensing raises privacy questions that a conventional temperature sensor

never would, forcing me to think beyond the standard encrypt-everything checklist to consider physicallayer eavesdropping risks. Each laboratory exercise mapped almost one-to-one onto a component of this project, which made the full course material feel far more concrete and applicable in hindsight.

14 Conclusion

This case study has demonstrated that WiFi Channel State Information captured from a pair of low-cost ESP32 microcontrollers can provide accurate, privacy-preserving, device-free human occupancy counting when embedded inside a properly layered IoT system. By tracing the complete pipeline from the physical WiFi channel through embedded firmware, REST and WebSocket protocols, cloud hosting on AWS EC2, and a real-time Chart.js dashboard, the project shows that the Device-Network-Cloud architecture, protocol selection criteria, and security principles taught throughout this course apply directly to a state-of-the-art sensing technique and not merely to textbook examples. Real challenges remain around cross-environment model generalisation, physical-layer privacy leakage, and multi-room scalability, but the rapid trajectory of the field culminating in the 2024 IEEE 802.11bf standard suggests these limitations will steadily be addressed. More broadly, this project reinforces why IoT matters today: it is the connective tissue that transforms an isolated clever sensing trick into a living system that can genuinely save energy, improve safety, and respect the privacy of every person it serves.

References

[1] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, “Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications,” *IEEE Communications Surveys*, vol. 17, no. 4, pp. 2347–2376, 2015.

[2] S. Yousefi, H. Narui, S. Dayal, S. Ermon, and S. Valaee, “A Survey on Behavior Recognition Using WiFi CSI,” *IEEE Communications Magazine*, vol. 55, no. 10, pp. 98–104, 2017.

[3] I. Sobron, J. Del Ser, I. Eizmendi, and M. Velez, “Device-Free People Counting in IoT Environments,” *IEEE IoT Journal*, vol. 5, no. 6, pp. 4396–4408, 2018.

[4] S. M. Hernandez, “ESP32-CSI-Tool,” GitHub.Available: https://github.com/ StevenMHernandez/ESP32-CSI-Tool.

[5] Espressif Systems, “esp-csi,” GitHub. Available: https://github.com/espressif/esp-csi.

[6] J. Strohmayer and M. Kampel, “Directional Antenna Systems for Long-Range Through-Wall Human Activity Recognition,” *arXiv preprint* arXiv:2401.01388, 2024.

[7] T. Kitagishi et al., “Wi-Monitor: WiFi CSI Crowd Counting with Low-Cost IoT Devices,” in *GIoTS 2022*, LNCS vol. 13533, Springer.

[8] H. Choi et al., “Wi-CaL: WiFi Sensing and ML Based Device-Free Crowd Counting,” *IEEE Access*, vol. 10, pp. 24395–24410, 2022.

[9] F. Meneghello et al., “Toward Integrated Sensing in IEEE 802.11bf,” *IEEE Comm. Magazine*, vol. 61, no. 7, pp. 128–133, 2023.

[10] T. Ropitault et al., “IEEE 802.11bf WLAN Sensing Procedure,” *IEEE Comm. Standards Magazine*, vol. 8, no. 1, pp. 58–64, 2024.

[11] X. Liu et al., “A Survey on Secure WiFi Sensing Technology,” *Sensors*, vol. 25, no. 6, p. 1913, 2025.

[12] H. J. Jara Ochoa et al., “Power Consumption of MQTT vs HTTP in IoT Platforms,” *Sensors*, vol. 23, no. 10, p. 4896, 2023.